第 1 课：你好 C++

一、课程信息

项目	内容
课次	第 1 课 / 共 40 课
主题	你好 C++——认识编程世界
适用对象	小学四年级，零编程基础
课时	100 分钟
教学环境	Dev-C++

教学目标（可检验）

1. 学生能**独立打开 Dev-C++**，新建源文件、编写代码、编译并运行，看到黑色窗口输出结果。
2. 学生能**背诵并默写** `cout` 输出语句的基本格式（含 `#include<iostream>`、`using namespace std;`、`int main()`、`return 0;`）。
3. 学生能用 `cout` 和 `endl` **输出自己的名字和一个简单的文字图形**（如小房子、小树）。

二、教学重点与难点

	内容
重点	① Dev-C++ 的基本操作流程（新建编写编译运行）；② `cout <<` 输出语句的写法
难点	① 理解"编译"的含义（为什么不能直接运行）；② 分号、引号、尖括号等符号的准确输入（四年级孩子对键盘符号不熟）

三、课堂时间分配（100 分钟）

时间段	时长	内容
0′–5′	5 min	**开场破冰**：自我介绍 + "什么是编程"
5′–25′	20 min	**讲解①**：认识 Dev-C++ + 第一个程序 Hello World
25′–35′	10 min	**练习①**：学生动手敲出 Hello World 并成功运行
35′–55′	20 min	**讲解②**：cout 详解 + 输出多行 + endl 换行
55′–65′	10 min	**练习②**：输出自己的名字和班级信息
65′–70′	5 min	**休息**
70′–85′	15 min	**讲解③**：用 cout 画文字图形（小房子）
85′–95′	10 min	**练习③**：自己设计一个小图形并输出
95′–100′	5 min	**总结 + 布置作业 + 下节课预告**

四、讲稿提纲

4.1 开场破冰（0′–5′）

> **老师说：**
> 同学们好！从今天开始，我们要学一门超级厉害的"魔法语言"——C++。
> 你们玩过游戏吧？《我的世界》玩过吗？那些游戏都是程序员用编程语言"写"出来的。**编程就是告诉电脑该做什么的一种方式**，就好像你给电脑写一封信，电脑读懂了就会照着做。
> 不过呢，电脑比较"笨"，它只认识一种特定格式的信。我们今天就来学怎么写这封信，让电脑乖乖听话！
> C++ 是一种很强大的编程语言，全世界的信息学竞赛——就是"编程比赛"——基本都用它。学会了 C++，以后你可以参加比赛拿奖，也可以自己做游戏、做机器人！
> …

4.2 讲解①：认识 Dev-C++ + Hello World（5′–25′）

环节 A：认识 Dev-C++

> **老师说：**
> 我们写作文用什么？用笔和纸，对吧？写代码也需要一个"本子"，这个本子叫做 **Dev-C++**，它是我们的**编程工具**。
> 现在请大家跟我一起操作——

【操作演示，同步带学生做】

1. 双击桌面上的 **Dev-C++** 图标，打开软件。
2. 点击菜单栏 **文件 新建 源代码**（或按快捷键 `Ctrl + N`）。
3. 看到一个白色的编辑区域——这就是我们写代码的地方。

> **老师说：**
> 大家注意，写完代码以后，要先**保存**（Ctrl + S），给文件起个名字，后缀名是 `.cpp`（比如 `hello.cpp`）。然后按 **F11**（编译并运行），电脑就会读懂我们的信，把结果显示出来。

环节 B：第一个程序——Hello World

> **老师说：**
> 程序员有个传统：学任何新语言，第一个程序永远是让电脑说"Hello World"（你好，世界）。我们也来试试！

【边讲边在大屏幕上敲代码】

```
#include<iostream>
using namespace std;
int main(){
    cout << "Hello World!" << endl;
    return 0;
}
```

> **老师逐行讲解：**
> 我们来一行一行看这封"信"：
> **第 1 行：** `#include<iostream>`
> - 这就像你做手工之前，先要**把工具箱搬过来**。`iostream` 就是一个工具箱，里面装着"输出"和"输入"的工具。没有这行，电脑就找不到工具，不知道怎么说话。
> **第 2 行：** `using namespace std;`
> - 这行现在不用深究，你就把它当成一句**"开场白"**，每次都要写上。就像写信要写"亲爱的某某某"一样，这是固定格式。
> **第 3 行：** `int main(){`
> - `main` 就是"主要的"意思。电脑拿到你的信，**永远从 main 这里开始读**。你可以把 `main(){}` 想象成一扇大门，电脑从这扇门进去，按顺序执行里面的命令。
> **第 4 行：** `cout << "Hello World!" << endl;`
> - 这是今天最重要的一行！
> - `cout` （读作 "C-out"）就是让电脑**说话**。`cout` = "C 语言的输出"。
> - `<<` 是**"把右边的东西送出去"**，你可以想象成一个传送带 ，把文字送到屏幕上。
> - `"Hello World!"` 是我们想让电脑说的话，**必须用英文双引号包起来**。
> - `endl` 是**换行**，就像写作文写完一段要按回车换到下一行。
> - **最后有个分号 ；**——这非常重要！每一句命令写完都要加分号，就像每句话结束要加**句号**一样。
> **第 5 行：** `return 0;`
> - 这句话的意思是**"报告，任务完成！"** 电脑执行到这里就知道程序结束了。
> **老师强调：**
> 大家记住，每个 C++ 程序都有这个**"骨架"**：
> ```
> # include
>
> using namespace std;
> int main(){
> // 在这里写你的命令
> return 0;
> }
> ```
> 以后每次写程序，先把骨架打好，再往里面填内容，就像先画好格子再写字一样。

4.3 练习①：动手敲 Hello World（25′–35′）

> **老师说：**
> 好，现在轮到你们了！请大家自己敲一遍刚才的代码，然后按 F11 运行。
> 看到黑色窗口显示"Hello World!"的同学举手！

【老师巡堂，帮助卡住的孩子，重点检查：】
- 是否漏了分号
- 引号是否用了中文引号（常见错误！）
- `iostream` 是否拼对
- 是否按了 F11 编译运行

4.4 讲解②：cout 详解 + 多行输出 + endl（35′–55′）

环节 A：输出多行

> **老师说：**
> 刚才我们只让电脑说了一句话。如果想让电脑说好几句话呢？有两种写法。

写法一：多个 cout

```
#include<iostream>
using namespace std;
int main(){
    cout << "我叫小明" << endl;
    cout << "我今年10岁" << endl;
    cout << "我喜欢编程" << endl;
    return 0;
}
```

运行结果：

```
我叫小明
我今年10岁
我喜欢编程
```

写法二：一个 cout 连续输出

```
#include<iostream>
using namespace std;
int main(){
    cout << "我叫小明" << endl << "我今年10岁" << endl << "我喜欢编程" << endl;
    return 0;
}
```

> **老师说：**
> 两种写法效果一样。初学阶段建议用**写法一**，一句话一个 `cout`，看起来更清楚。

环节 B：endl 的作用

> **老师说：**
> 如果我把 `endl` 去掉，会怎样呢？我们来试试：

```
#include<iostream>
using namespace std;
int main(){
    cout << "AAA";
    cout << "BBB";
    cout << "CCC";
    return 0;
}
```

运行结果：

```
AAABBBCCC
```

> **老师说：**
> 看到了吗？全挤在一起了！ `endl` 就像按了一下**回车键**，让下一个内容换到新的一行。记住：**想换行就加 `endl`，不想换行就不加。**

环节 C：输出数字

> **老师说：**
> `cout` 不只能输出文字，还能输出数字哦！而且数字**不用加引号**。

```
#include<iostream>
using namespace std;
int main(){
    cout << 100 << endl;
    cout << 3 + 5 << endl;
    cout << "3 + 5 = " << 3 + 5 << endl;
    return 0;
}
```

运行结果：

```
100
8
3 + 5 = 8
```

> **老师说：**
> 注意看第三行，我们可以**把文字和数字混在一起输出**，用 `<<` 连接就行。引号里的内容原样输出，不加引号的数字会被电脑算出结果。

4.5 练习②：输出自己的信息（55′–65′）

> **任务：** 写一个程序，输出以下内容（换成你自己的真实信息）：
> `姓名：张三 年龄：10 学校：阳光小学 爱好：踢足球`

【老师巡堂指导】

4.6 休息（65′–70′）

> 大家站起来活动活动，休息 5 分钟！

4.7 讲解③：用 cout 画文字图形（70′–85′）

> **老师说：**
> 现在我们来玩一个好玩的——用代码**画画**！虽然是黑色窗口，但我们可以用符号拼出图形。

例题 1：画一个小房子

```
#include<iostream>
using namespace std;
int main(){
    cout << "    *" << endl;
    cout << "   ***" << endl;
    cout << "  *****" << endl;
    cout << " *******" << endl;
    cout << "*********" << endl;
    cout << " |     |" << endl;
    cout << " |     |" << endl;
    cout << " |     |" << endl;
    cout << " +-----+" << endl;
    return 0;
}
```

运行结果：

```
    *
   ***
  *****
 *******
*********
 |     |
 |     |
 |     |
 +-----+
```

> **老师说：**
> 是不是像一个小房子？上面是三角形的屋顶，下面是方形的墙壁。秘诀就是**用空格控制位置**。
> 注意：每一行 `cout` 输出的内容里，**前面的空格数量不一样**，这样就能让星号排成三角形。

例题 2：画一棵小树

```
#include<iostream>
using namespace std;
int main(){
    cout << "    *" << endl;
    cout << "   ***" << endl;
    cout << "  *****" << endl;
    cout << " *******" << endl;
    cout << "*********" << endl;
    cout << "    |" << endl;
    cout << "    |" << endl;
    cout << "  -----" << endl;
    return 0;
}
```

运行结果：

```
    *
   ***
  *****
 *******
*********
    |
    |
  -----
```

> **老师说：**
> 这棵树和房子的区别在哪？下半部分不同！树干是细的 `|`，底下是一条横线当泥土。
> 你们看，只要你有想象力，用这些简单的符号就能画出很多东西！

4.8 练习③：自己设计小图形（85′–95′）

> **任务：** 发挥想象力，用 `cout` 画一个你喜欢的图形。可以是：
> - 一颗爱心
> - 一个菱形
> - 一把雨伞
> - 你名字的首字母（用星号拼出来）
> - 任何你想画的东西！
>
> 画好了举手给老师看，最有创意的同学有小奖品！

【爱心参考（如果有学生想画）：】

```
#include<iostream>
using namespace std;
int main(){
    cout << "  **   **" << endl;
    cout << " **** ****" << endl;
    cout << "**********" << endl;
    cout << " ********" << endl;
    cout << "  ******" << endl;
    cout << "   ****" << endl;
    cout << "    **" << endl;
    cout << "     *" << endl;
    return 0;
}
```

4.9 总结 + 布置作业 + 预告（95′–100′）

> **老师说：**
> 今天我们学了三件事：
> 1. **认识 Dev-C++**：新建文件、写代码、按 F11 运行。
> 2. **程序骨架**：每个程序都要有 `#include<iostream>`、`using namespace std;`、`int main() {}`、`return 0;`。
> 3. **cout 输出**：用 `cout << "内容" << endl;` 让电脑说话、换行，还能输出数字和画图形。
> …

七、常见错误与应对

序号	常见错误	现象	应对方法
1	**用了中文标点**	编译报错，错误信息一大堆	告诉学生：代码里的**所有标点都要用英文**！分号 `;`、引号 `""`、尖括号 `<>` 都是英文的。可以教学生切换输入法的方法。
2	**忘记写分号**	编译报错，提示 `expected ';'`	比喻：每句命令后面的分号就像**句号**，没有句号电脑就不知道这句话说完了。
3	**`iostream` 拼写错误**	编译报错，提示找不到头文件	带学生一起拼读：`i-o-s-t-r-e-a-m`，"输入(i)输出(o)流(stream)"。
4	**`main` 写成 `mian` 或 `Main`**	编译报错或无法运行	C++ 区分大小写！`main` 全部小写。可以让学生记：**m-a-i-n，"主要的"。**
5	**双引号只写了一半**	编译报错	提醒：引号必须**成对出现**，左边一个右边一个，就像括号一样。
6	**没有保存文件就运行**	Dev-C++ 可能运行旧版本代码	养成习惯：**先 Ctrl+S 保存，再 F11 运行**。
7	**文件后缀名不对**	保存时没有 `.cpp` 后缀	第一次保存时演示：文件名填 `hello.cpp`，确认后缀正确。

九、下节课预告

> **老师说：**
> 今天我们学会了让电脑"说话"。但是现在电脑只会说我们事先写好的话，就像一个只会背台词的演员。
> 下节课，我们要教电脑**"听话"**——让电脑能接收**你从键盘输入的内容**，然后做出回应。比如你输入自己的名字，电脑就会说"你好，XXX！"。是不是很酷？
> 这就是——**变量与输入**！下周见！

附录：板书/PPT 要点提炼

```
┌──────────────────────────────────────────┐
│            第 1 课：你好 C++              │
├──────────────────────────────────────────┤
│                                          │
│  程序骨架（每次必写）：                    │
│                                          │
│    #include<iostream>                    │
│    using namespace std;                  │
│    int main(){                           │
│        // 你的代码写在这里                 │
│        return 0;                         │
│    }                                     │
│                                          │
├──────────────────────────────────────────┤
│                                          │
│  输出语句：                               │
│    cout << "内容" << endl;                │
│                                          │
│  要点：                                   │
│     文字要加双引号 ""                     │
│     数字不用加引号                        │
│     endl = 换行                           │
│     每句结尾加分号 ;                      │
│     所有符号用英文!                       │
│                                          │
├──────────────────────────────────────────┤
│                                          │
│  Dev-C++ 操作：                           │
│    新建：Ctrl + N                         │
│    保存：Ctrl + S （文件名 xxx.cpp）      │
│    运行：F11                              │
│                                          │
└──────────────────────────────────────────┘
```